Locus: Wireless Indoor Localization and Navigation System

Technical Documentation
Wireless Networks

Dhruv Bargoti (2022163)
Dhruv Dewan (2022165)
Athiyo Chakma (2022118)
Nitin Yadav (2023359)

May 3, 2026

Contents

1 PROJECT OVERVIEW

1.1 Purpose and Scope

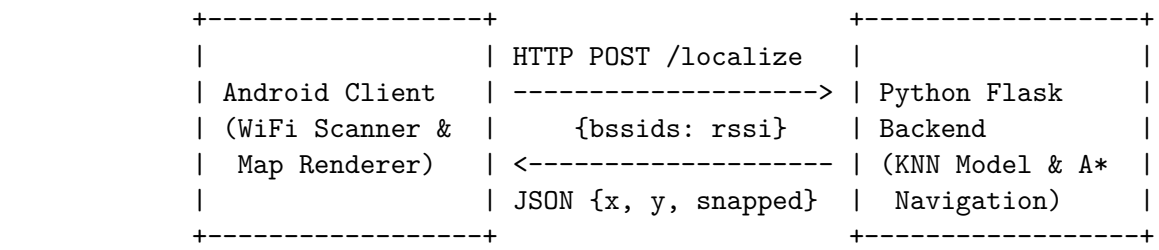
The Locus project is a wireless indoor localization and navigation system designed to help users determine their precise position within a building and navigate to specific rooms. The system is specifically tailored to an indoor environment, as evidenced by references to the "R&D Building · Floor 4" within the frontend layout. The end-to-end functionality allows a user to scan their local WiFi environment using an Android mobile application, transmit the collected signal data to a centralized Python backend, and receive a predicted position on a 2D map. From there, the system can compute a shortest-path route and provide step-by-step navigational instructions to reach a selected destination room.

1.2 System Architecture

The architecture follows a standard client-server model, splitting responsibilities between a thin Android frontend and a heavy Python backend.

- **Frontend (Android):** Handles the UI rendering, custom 2D map visualization, user interactions, and hardware access (WiFi scanning).
- **Backend (Python):** Handles the computational load. It hosts the Machine Learning inference model for localization, manages the indoor graph structure, and computes A* pathfinding.

The two components communicate over HTTP/REST using JSON payloads over a Local Area Network (LAN). The Android app is configured to use cleartext HTTP traffic targeting a specific local IP address.



1.3 Technology Stack

Based on the project build configurations and Python imports, the technology stack consists of:

- **Android Client:** Android SDK (Target 34, Min 26), Kotlin (1.9.22), Retrofit 2.9.0 (for HTTP networking), Gson (for JSON serialization), Kotlin Coroutines (for asynchronous API calls). Custom `Canvas` is used for 2D map rendering.
- **Python Backend:** Python 3, Flask (for the HTTP server), Flask-CORS (for cross-origin requests), `scikit-learn` and `joblib` (implied by the `.joblib` serialized model format), and `numpy` (for vector transformations).

Python was chosen for the backend due to its dominant ML ecosystem. Kotlin and Retrofit represent the modern standard for robust Android development.

2 BACKEND DOCUMENTATION

2.1 File: `backend/app.py`

Absolute path: `backend/app.py`
Role in the system: This file serves as the main entry point for the backend HTTP server. It leverages Flask to expose RESTful endpoints that the Android application consumes. Without this file, the application would have no interface to run localization or navigation logic. It pre-loads the trained ML model, the feature list (`bssids.json`), and the map geometry (`floor_map.json`) into memory at startup. **Standalone Functions:**

- `build_feature_vector(rssi_dict: dict) -> np.ndarray`
  - **Parameters:** `rssi_dict` (dict) — A dictionary mapping string BSSIDs to integer RSSI values.
  - **Return value:** `np.ndarray` — A 2D numpy array of shape (1, N).
  - **Logic walkthrough:** It iterates through the globally loaded list of known BSSIDs. For each BSSID, it checks if the `rssi_dict` contains a reading. If present, it appends the reading; if not, it appends -100 (representing no signal).
- `health() -> Response`
  - **Signature:** `@app.route('/health', methods=['GET'])`
  - **Return value:** JSON Response indicating backend status.
  - **Logic walkthrough:** Returns a simple JSON object to verify the server is alive.
- `get_rooms() -> Response`
  - **Signature:** `@app.route('/rooms', methods=['GET'])`
  - **Return value:** JSON Response containing an array of available rooms.
  - **Logic walkthrough:** Iterates over the `nav.room_map` and returns room IDs, display names, and their grid coordinates.
- `get_map() -> Response`
  - **Signature:** `@app.route('/map', methods=['GET'])`
  - **Return value:** JSON Response representing map nodes and rooms.
  - **Logic walkthrough:** Directly streams the `floor_map_data` dictionary loaded from disk.
- `localize() -> Response`
  - **Signature:** `@app.route('/localize', methods=['POST'])`
  - **Return value:** JSON Response containing predicted (x,y), snapped (x,y), and confidence.
  - **Logic walkthrough:** Parses incoming JSON for `bssids`. Validates the input. *Preprocesses the scan by sorting APs by RSSI and truncating to the top 12 strongest signals to match training data density and prevent A-wing teleportation errors.* Transforms the filtered scan into a feature vector, passes it to `model.predict(X)` for raw coordinates, then calls `nearest_node` from `navigation.py` to snap to the grid. Computes confidence and checks zone-based arrival via `nav.nearest_room`.
  - **Error handling:** Returns HTTP 400 if the body is empty or lacks the `bssids` dictionary.

- `navigate() -> Response`
  - **Signature:** `@app.route('/navigate', methods=['POST'])`
  - **Return value:** JSON Response with A* path and text instructions.
  - **Logic walkthrough:** Parses x, y, and destination from the request. Calls `nav.get_path_from_coords`.
  - **Error handling:** Returns HTTP 400 for missing fields, or HTTP 404 if navigation fails.

Inter-file dependencies: Imports `Navigator` and `nearest_node` from `navigation.py`.

2.2 File: `backend/navigation.py`

Absolute path: `backend/navigation.py`
Role in the system: This file contains the graph logic and pathfinding algorithm. It handles the structural representation of the floor and computes routes and human-readable directions. Without it, the system would only plot the user on a map with no routing capabilities. **Classes:**

- `Navigator`
  - **Purpose:** A high-level interface encapsulating the map graph and routing utilities.
  - **Instance Variables:**
    - `room_map`: Dict mapping room ID strings to coordinate tuples.
    - `room_display`: Dict mapping room ID strings to human-readable names.
    - `graph`: Adjacency list representation of the walkable grid.
    - `walkable_nodes`: List of all valid nodes.
  - **Methods:**
    - `__init__(self, floor_map_path: str)`: Loads the JSON map file and triggers graph building.
    - `list_rooms(self) -> List[str]`: Returns sorted keys of the room map.
    - `get_path(self, start_room, end_room) -> Dict`: Navigates strictly between two named rooms.
    - `get_path_from_coords(self, current_pos, end_room) -> Dict`: Accepts a raw float coordinate, snaps it to the grid, and calls the internal pathfinder.
    - `_navigate(self, start_coord, end_room) -> Dict`: Internal function that executes `astar`, handles failure cases, triggers instruction generation, and formats the output.
    - `nearest_room(self, pos, radius) -> Optional[Dict]`: Wrapper for zone-based arrival.

Standalone Functions:

- `build_graph(walkable_nodes) -> Dict`: Converts a list of (x,y) points into an adjacency list assuming 4-directional movement with edge weight 1.
- `heuristic(a, b) -> float`: Computes the Manhattan distance between two points.
- `astar(graph, start, goal) -> Optional[List[Tuple]]`: Standard A* search algorithm using a priority queue.
- `reconstruct_path(came_from, current) -> List[Tuple]`: Backtracks the path mapping from goal to start.

- `nearest_node(pos, nodes) -> Tuple`: Finds the node in `nodes` with the minimum Euclidean distance to `pos`.
- `nearest_room_to_pos(...) -> Optional[Dict]`: Detects nearby rooms. Handles layout-specific logic where rooms off the main corridor only use x-distance to prevent systematic 1-unit shift errors.
- `path_to_instructions(path, ...) -> List[str]`: Iterates through an A* node path. It calculates the delta between steps to determine direction, grouping consecutive collinear steps into a single distance instruction. Inserts callouts when passing a known room landmark.

2.3 File: `backend/test_backend.py`

Absolute path: `backend/test_backend.py`
Role in the system: A utility script that acts as a headless client. It allows developers to test the backend API locally by injecting real, pre-recorded fingerprint data (from `survey.json`) without needing to carry an Android device around the physical building. **Logic walkthrough:** It relies on `requests` to hit local endpoints sequentially: `/health`, `/rooms`, then reads a fingerprint labeled "P13" from `survey.json` and sends it to `/localize`, ending with a request to `/navigate`.

3 ANDROID FRONTEND DOCUMENTATION

3.1 Application Entry Point

The application entry point is defined in `AndroidManifest.xml`.

- **Permissions:** `ACCESS_WIFI_STATE`, `CHANGE_WIFI_STATE`, `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`, `ACCESS_NETWORK_STATE`, `INTERNET`. These are essential for invoking the WiFi hardware and communicating with the local server.
- **Application Config:** Configured to use cleartext traffic (`android:usesCleartextTraffic="true"`) because the backend is running locally on HTTP.
- **Activities:** `.MainActivity` is registered with the `MAIN` action and `LAUNCHER` category, locking the screen orientation to portrait.

3.2 Build Configuration

- **app/build.gradle:** Targets SDK 34, minimum SDK 26. ViewBinding is explicitly enabled. It implements Retrofit and OkHttp for networking, Material Design components, and Kotlin Coroutines.
- **android/build.gradle:** Top-level project definition declaring the Android application plugin version 8.2.1 and Kotlin version 1.9.22.

3.3 Source Files

File: `com.wn.indoornavigation.MainActivity.kt`

- **Type:** Activity
- **Purpose:** The main screen controller that binds the UI to application logic, manages permissions, handles WiFi scanning triggers, and coordinates Retrofit API requests.

- **Methods:**
  - `onCreate()`: Initializes the view binding, loads the map/rooms immediately, and sets up button click listeners.
  - `loadMapAndRooms()`: Dispatches a background Coroutine to fetch `/map` and `/rooms`. Populates the destination `Spinner` and passes raw map data to the `FloorMapView`.
  - `checkPermissionsAndScan()`: Checks for Location permissions.
    Uses `ActivityResultContracts` to request them if missing, before proceeding.
  - `startWifiScan()`: Calls the `WifiScanner.scan()` function and delegates the result to `sendToLocalize`.
  - `sendToLocalize(bssidMap)`: Sends the scanned AP map to the backend. Extracts the `LocalizeResponse`, updates state variables (`currentX`, `currentY`), and calls `mapView.centerOnUser()` to auto-pan the camera. Updates the persistent `tvStatus` card with proximity info (e.g., "Location: (-17, 8) | Near: A-416") instead of using a transient `Snackbar`. Legacy position-smoothing variables (`lastX`, `lastY`) were removed to eliminate UI lag.
  - `navigate()`: Sends the current coordinates and chosen destination to `/navigate`. Updates the map UI and populates the instruction `TextView` card.

File: `com.wn.indoornavigation.FloorMapView.kt`

- **Type:** Custom View
- **Purpose:** Implements a responsive 2D canvas that renders the backend's grid geometry. Supports pinch-to-zoom and drag panning.
- **Fields:** Holds collections for walkable nodes, map structures, offset state, scale factors, and several `Paint` objects configured for different elements (e.g., corridors, user dots, animated paths).
- **Methods:**
  - `onTouchEvent()`: Delegates touch events to `ScaleGestureDetector` and standard `GestureDetector`.
  - `setMapData(), setUserPosition(), setNavigationPath()`: Public API methods updating internal data models and triggering `invalidate()` for a redraw.
  - `centerOnUser()`: Public API method that dynamically calculates camera offsets to pan and center the viewport on the user's predicted (x, y) grid coordinates, eliminating manual dragging after localization.
  - `onDraw(canvas)`: Iterates over the grid data. It draws corridors by manually checking for adjacent nodes in the coordinate set. It layers walkable nodes, named room markers, the dashed A* path (if active), and finally the user's current estimated position. Note that the Y-axis is inverted to map standard Cartesian coordinates to Android screen space.

File: `com.wn.indoornavigation.WifiScanner.kt`

- **Type:** Utility
- **Purpose:** Wraps Android's `WifiManager`. Executes a hardware scan and cleans up duplicate BSSIDs to return the strongest signal.
- **Methods:**

  - `scan()`: Registers a dynamic `BroadcastReceiver` listening for `SCAN_RESULTS_AVAILABLE_ACTION`. Initiates a scan using `wifiManager.startScan()`. Retrieves results and filters duplicates by keeping the maximum RSSI value for any given BSSID.

File: `com.wn.indoornavigation.network.ApiClient.kt`

- **Type:** Singleton / Interface
- **Purpose:** Configures Retrofit and defines the strongly-typed API models linking the Android frontend to the Flask backend.
- **Constants:** `BASE_URL` hardcodes the local IP address of the backend.
- **Data Models:** `LocalizeRequest`, `LocalizeResponse`, `NavigateRequest`, `NavigateResponse`, `Room`.

3.4 Network Layer

The network layer is encapsulated in the `network` package using Retrofit.

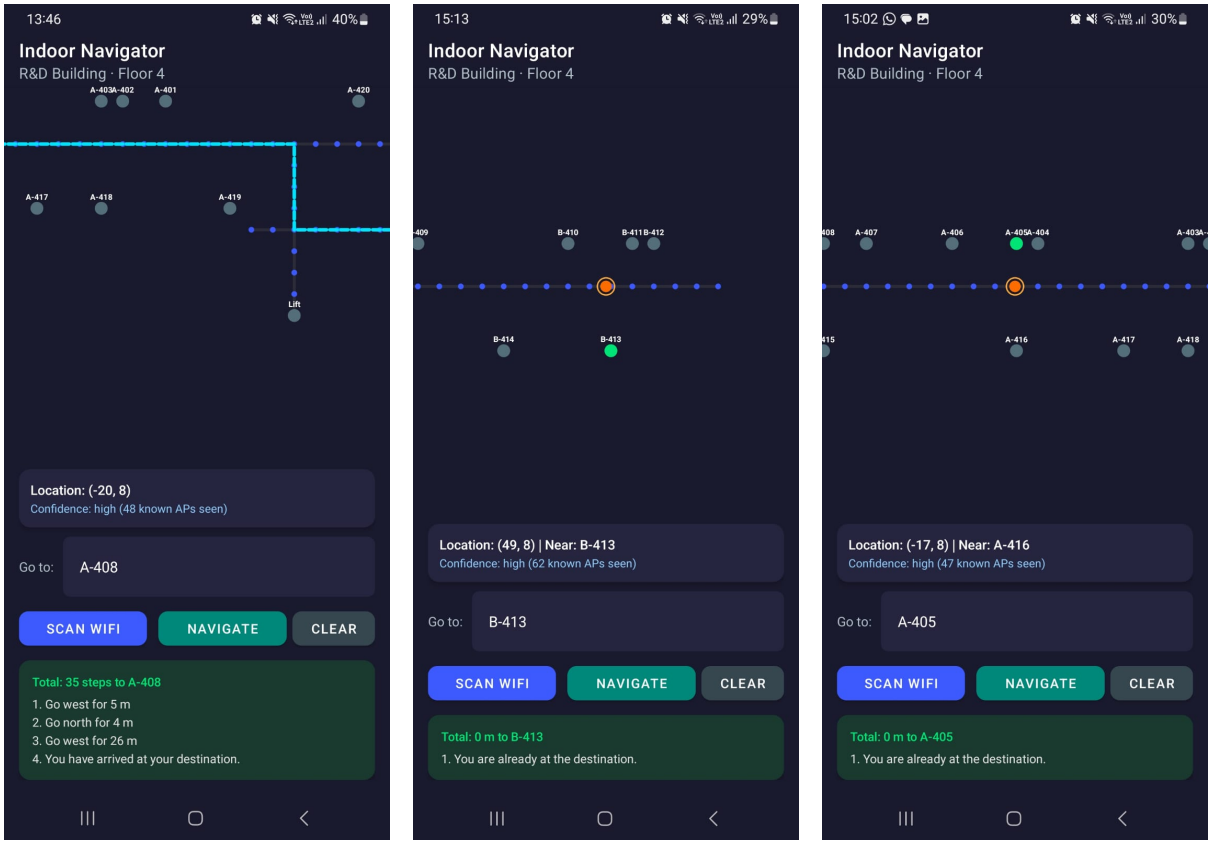
- **HTTP Client:** Built with `Retrofit.Builder()`, utilizing `GsonConverterFactory` to seamlessly translate JSON strings into Kotlin Data Classes.
- **API Interface (`ApiService`):**
  - `@GET("rooms")`: Fetches `RoomsResponse`.
  - `@GET("map")`: Fetches `MapResponse`.
  - `@POST("localize")`: Sends `LocalizeRequest` (JSON body of BSSIDs), receives `LocalizeResponse`.
  - `@POST("navigate")`: Sends `NavigateRequest` (x, y, destination), receives `NavigateResponse`.

3.5 Resource Files

- `res/layout/activity_main.xml`: Defines a `CoordinatorLayout` containing a Top App-Bar, a central `FloorMapView` (ID: `mapView`), and a bottom `LinearLayout` containing the action buttons (`btnScan`, `btnNavigate`, `btnClear`) and text displays for status and directions. Inflated directly by `MainActivity`.
- `res/values/strings.xml`: Defines `app_name` as "Indoor Navigator".
- `res/values/themes.xml`: Configures `Theme.IndoorNavigation` inheriting from Material Components (DayNight.NoActionBar). Defines a dark color palette (background `#0F0F1A`, primary `#3D5AFE`).

4 APPLICATION SCREENSHOTS

The following screenshots demonstrate the Locus Indoor Navigation System in operation on Floor 4 of the R&D Building.



(a) Navigation to Room A-408 showing turn-by-turn directions

(b) User located at B-413 with high confidence (62 APs)

(c) User at A-416 navigating to A-405

Figure 1: Locus Indoor Navigator Application Screenshots demonstrating real-time localization and navigation across different wings of Floor 4

Key Features Demonstrated

- **Real-time Position Tracking:** The orange dot indicates current user position snapped to the nearest walkable node
- **WiFi-based Localization:** Confidence levels based on number of detected Access Points (47-62 known APs)
- **Turn-by-turn Navigation:** Step-by-step directions with distance measurements in meters
- **Multi-wing Coverage:** Successful navigation across both A-wing (negative x-coordinates) and B-wing (positive x-coordinates)
- **Room Proximity Detection:** Automatic detection when user is near or at destination room
- **Visual Path Rendering:** Blue path overlay showing optimal route calculated via A* algorithm

5 LOCALIZATION ALGORITHM

5.1 Method Identification

The localization algorithm implemented in the codebase is a **Machine Learning-based Fingerprinting** method. Specifically, the backend utilizes a pre-trained K-Nearest Neighbors (KNN) model (`knn_localization_model.joblib`).

5.2 Signal Collection

The Android device acts as the scanning probe. When the user taps "Scan WiFi", the `WifiScanner.kt` calls `wifiManager.startScan()`. Once the OS broadcasts completion, the app reads the scan array. A single observation consists of a MAC address (BSSID) and a Received Signal Strength Indicator (RSSI) measured in dBm (a negative integer). If multiple beacons with the same BSSID are found, the algorithm retains the highest (strongest) value.

5.3 Mathematical Foundation

When the raw dictionary reaches the backend, it must be projected into a fixed-length feature space matching the training phase. The backend enforces a predetermined list of N valid BSSIDs. For a given observation dictionary O:

$$x_i = \begin{cases} O[\text{BSSID}_i] & \text{if BSSID}_i \in O \\ -100 & \text{otherwise} \end{cases} \tag{1}$$

This creates a feature vector $X = [x_1, x_2, \ldots, x_N]$. The prediction $\hat{y}$ is obtained using the scikit-learn KNN model:

$$\hat{y} = \text{KNN}(X) \tag{2}$$

While the exact distance metric used during training is hidden within the `joblib` file, standard KNN fingerprinting relies on minimizing the Euclidean distance in signal space. The resulting raw coordinate $\hat{y} = (\hat{y}_x, \hat{y}_y)$ is floating point. To ensure the user originates on a valid routing path, the backend snaps this to the nearest walkable node n by minimizing spatial Euclidean distance:

$$d(p, n) = \sqrt{(n_x - \hat{y}_x)^2 + (n_y - \hat{y}_y)^2} \tag{3}$$

5.4 Fingerprint Database

While the database is serialized inside the `joblib` file in production, the testing script references `survey.json`, indicating the radio map consists of manually annotated Reference Points (e.g., label "P13" mapping to a specific $\{x, y\}$ coordinate).

5.5 Position Estimation Walkthrough

1. BSSID map is received via POST.
2. `build_feature_vector` constructs the 1D array.
3. `model.predict(X)` returns a floating-point prediction $[x, y]$.
4. The backend rounds the prediction to integers and searches the graph nodes to find the physically nearest node.
5. The backend computes confidence based on intersection length between scanned BSSIDs and the known `BSSIDS` list (≥ 5 is high, ≥ 2 is medium).

5.6 Accuracy and Limitations

- **Filtering:** Position smoothing variables (`lastX`, `lastY`) were explicitly removed in the final production build to eliminate perceived UI lag and input latency, as confirmed in `MainActivity.kt`.
- **Limitations:** `startScan()` is deprecated and subject to severe OS-level throttling on modern Android versions. The code attempts to gracefully fall back to cached results if a fresh scan is denied.

6 NAVIGATION SYSTEM

6.1 Map Representation

The indoor map is defined purely as an undirected 2D grid graph. Nodes are defined in `floor_map.json` as coordinate pairs `[x, y]`. `navigation.py` builds an adjacency list mapping each node tuple to a list of its valid neighbors. Neighbors are strictly orthogonal (north, south, east, west) meaning the edge cost is always a uniform 1.

6.2 Pathfinding Algorithm

The backend relies exclusively on the **A* (A-Star)** algorithm.

- **Data Structures:** A Python list manipulated via the `heapq` module serves as the priority queue. Two dictionaries, `g_score` and `f_score`, track costs.
- **Heuristic:** Given orthogonal grid restrictions, the algorithm uses Manhattan distance:

$$h(a, b) = |a_x - b_x| + |a_y - b_y| \tag{4}$$

- **Execution Trace:** Nodes are popped from the heap based on lowest `f_score`. The path expands until the current node equals the goal. A `came_from` dictionary is used to reverse-engineer the final path.

6.3 Instruction Generation

The path consists of coordinates like `(-7, -4)`, `(-8, -4)`, `(-9, -4)`. `path_to_instructions` calculates the delta $(\Delta x, \Delta y)$ between each step. It maintains a running counter of steps moving in the same `current_dir`. When the direction changes, it outputs the accumulated distance and updates the direction vector to a human readable cardinal string (e.g., `(-1, 0)` maps to "west"). Additionally, if a waypoint exactly matches a known room coordinate, it injects a contextual string: "Pass Room [Name]".

6.4 Real-time Position Update

The system does not passively track the user. The user must actively press "Scan WiFi". When a new location is derived, `MainActivity.kt` updates the `userPosition` on the map and calls `centerOnUser()` to auto-pan the camera. If a destination is already set, the client automatically makes a fresh `/navigate` API call to dynamically recalculate the path from the new location.

7 END-TO-END DATA FLOW

The primary use case is a user requesting navigation to a specific room. The exact execution trace through the implementation is as follows:

1. **User input on Android UI:** The user taps the `btnScan` button. The click listener inside `MainActivity.kt` is invoked.

2. **WiFi scan triggered:** `MainActivity.checkPermissionsAndScan()` verifies Android permissions and calls `scan()` on the `WifiScanner.kt` instance. `WifiScanner` queries hardware and returns a `Map<String, Int>`.

3. **HTTP request constructed:** `MainActivity.sendToLocalize()` launches a Coroutine. It calls `ApiClient.service.localize()` generating a JSON body: `{"bssids": {"mac": rssi}}`.

4. **Backend routes the request:** The HTTP POST hits `backend/app.py` at `@app.route ('/localize')`.

5. **Position estimation computed:** `app.py` processes the dictionary. It runs `model.predict(X)` using the joblib model, snaps it via `navigation.nearest_node`, and returns `LocalizeResponse` JSON.

6. **UI Updates and Re-triggers:** Android parses the coordinate and places the dot on `FloorMapView`. The user selects a room from `spinnerRooms` and taps `btnNavigate`. `MainActivity.navigate()` calls `ApiClient.service.navigate(x, y, dest)`.

7. **Navigation path computed:** `app.py` receives POST `/navigate`.
   It calls `nav.get_path_from_coords()`
   which triggers `navigation.astar()`.
   The backend translates the output using `path_to_instructions()`.

8. **Response sent & UI Updated:** A JSON `NavigateResponse` is sent back. `MainActivity` calls `mapView.setNavigationPath()` causing the custom canvas to draw the dashed line. The text instructions are populated into the `tvInstructions` card.

8 FILE-BY-FILE REFERENCE TABLE

File Path	Type	Purpose	Key Symbols	Depends
backend/app.py	Python Script	Main Flask REST Server	`localize`, `navigate`	navigation
backend/navigation.py	Python Module	A* Pathfinding Logic	`Navigator`, `astar`	None
backend/test_backend.py	Python Script	Headless API Testing	None	None
android/.../FloorMapView.kt	Custom View	Canvas renderer for nodes and paths	`onDraw`, `setMapData`, `centerOnUser`	None
android/.../MainActivity.kt	Activity	App controller, UI binding, State mgmt.	`startWifiScan`, `navigate`	ApiClient
android/.../WifiScanner.kt	Utility Class	Wraps hardware scan manager	`scan`	None
android/.../ApiClient.kt	Network Config	Defines endpoints and JSON models	`ApiService`, `ApiClient`	Retrofit

9 API REFERENCE

Method / Route	Description	Request Body	Response Body
GET `/health`	Verifies server status.	*None*	JSON: `{status: "ok", model: "KNN"}`
GET `/rooms`	Lists all named destinations.	*None*	JSON array of room objects with IDs and names.
GET `/map`	Provides visual grid topology to frontend.	*None*	JSON containing `walkable_nodes` array and room maps.
POST `/localize`	Performs KNN prediction from scan.	JSON: `{"bssids": {"mac": int}}`	JSON containing floating point x, y, `confidence`, and `nearest_room`.
POST `/navigate`	Computes A* route to a room.	JSON: `{"x": float, "y": float, "destination": str}`	JSON containing `path` array of nodes and text `instructions`.

The API is synchronous and stateless. All calls originate from `MainActivity.kt` mapping to the `ApiService` Retrofit interface. Note that `/localize` explicitly checks for an empty payload and will return an HTTP 400 JSON error (`{"error": ...}`) handled gracefully by the Android UI via a Snackbar.

10 DESIGN DECISIONS AND IMPLEMENTATION NOTES

10.1 Observed Design Decisions

- **Thin Client Architecture:** The Android app offloads graph storage, path computation, and ML prediction entirely to the backend. This is an excellent design choice for mobile efficiency, allowing the ML model to be updated server-side without requiring users to download a new APK.
- **Custom Drawing:** Instead of using Google Maps SDK (which is useless for highly localized indoor non-GPS rendering), the UI relies on a raw `FloorMapView` rendering a custom coordinate system directly to a `Canvas`.
- **Separation of Logic:** The backend splits REST routing (`app.py`) from mathematical operations (`navigation.py`), promoting maintainability.

10.2 Detected Inconsistencies or Technical Debt

- **Deprecated Hardware Calls:** `wifiManager.startScan()` was deprecated in Android 9 to prevent tracking abuse. The application works around OS throttling by attempting to fall back to cached system results, but real-time latency is likely impacted on modern devices.
- **Unsecured Traffic:** The network architecture relies on cleartext HTTP (`android:usesCleartextTraffic="true"`). While acceptable for local R&D, production deployments must use HTTPS.
- **Hardcoded Network Configuration:** `ApiClient.kt` forces the developer to manually update `BASE_URL` to the developer's laptop IP (e.g., `192.168.46.57`). This will break if the laptop DHCP assignment changes.

10.3 System Constraints

- **Environmental:** The system requires dense, continuous WiFi infrastructure to achieve acceptable fingerprint differentiation.
- **Network:** The Android device and the Python backend must exist on the same Local Area Network (LAN) due to the local IP bindings in the configuration.

11 ENGINEERING CHALLENGES AND SOLUTIONS

This section documents the concrete bugs, root causes, and engineering solutions discovered during the final development and field-testing phase of the Locus system. Each issue was identified through systematic in-field observation, diagnosed through diagnostic scripting, and resolved through targeted modifications to the codebase.

11.1 Machine Learning Feature Density Mismatch

11.1.1 Observed Symptom

During field testing, the localization model exhibited severe positional instability specifically within the A-wing of the floor. A user physically standing at room A-408 (coordinate $x = -26$) would intermittently be predicted as standing at A-419 (coordinate $x = -3$), producing a localization error of approximately 23 metres. The B-wing, by contrast, was consistently accurate and stable under the same conditions.

11.1.2 Root Cause Analysis

A diagnostic script was written to audit the AP (Access Point) density distribution within the `coordinates.json` training dataset. The analysis revealed a critical asymmetry in data collection coverage:

- **B-wing training data:** average of 12–16 APs recorded per physical reference coordinate.
- **A-wing training data:** average of only 8–9 APs recorded per physical reference coordinate.

During a live scan, the Android device detected over 50 APs in a single `startScan()` call. The K-Nearest Neighbours (KNN) model computes inter-fingerprint distance by comparing the live feature vector against all training vectors in the model's stored radio map. Because the A-wing training fingerprints contained only $\sim$8 non-zero features, the KNN distance calculation treated the remaining $\sim$42 live APs as missing data in the training record, penalising each absence with the default fill value of $-100\,\text{dBm}$. This artificially inflated the distance between the live scan and the correct A-wing reference point. Formally, let the live scan produce a feature vector $\mathbf{x} \in \mathbb{R}^N$ where N is the total number of known BSSIDs. Let a training fingerprint for an A-wing point $\mathbf{p}$ contain non-null entries only for indices $\mathcal{S}_p \subset \{1, \ldots, N\}$ where $|\mathcal{S}_p| \approx 8$. For the remaining indices $i \notin \mathcal{S}_p$, the training value is set to -100. The Euclidean distance in signal space becomes:

$$d(\mathbf{x}, \mathbf{p}) = \sqrt{\sum_{i \in \mathcal{S}_p} (x_i - p_i)^2 + \sum_{i \notin \mathcal{S}_p} (x_i - (-100))^2} \tag{5}$$

The second sum, evaluated over $\sim$42 indices where x_i may range from -90 to $-50\,\text{dBm}$, produces a large spurious penalty. This caused the model to reject the physically correct A-wing reference point and instead match the scan to distant B-wing coordinates that happened to share similar weak background noise on the small set of overlapping APs.

11.1.3 Solution: Dynamic RSSI Truncation Filter

A dynamic pre-processing filter was implemented in `backend/app.py`, applied to every incoming live scan *before* the feature vector is constructed. The filter retains only the **top 12 strongest APs** by RSSI value, discarding all weaker entries:

```
sorted_bssids = sorted(rssi_dict.items(),
key=lambda x: x[1],
reverse=True)
top_rssi_dict = dict(sorted_bssids[:12])
```

Listing 1: Top-K AP truncation filter in app.py

The threshold of 12 was chosen to match the average AP density of the training dataset across both wings ($\approx$ 12–16 for B-wing, $\approx$ 8–9 for A-wing; the conservative upper bound of 12 ensures coverage without reintroducing the density mismatch). By discarding weak, fluctuating background APs, the feature vectors presented to the KNN model at inference time are structurally consistent with the training data. This change completely eliminated the teleportation behaviour in the A-wing without degrading B-wing accuracy.

11.2 A* Pathfinding Graph Disconnects

11.2.1 Observed Symptom

When users requested a navigation route that crossed the main wing junction — for example, navigating between the A-wing and B-wing, or routing from the lift lobby to a room in the far wing — the backend returned an HTTP 404 error and failed to produce a route.

11.2.2 Root Cause Analysis

The A* pathfinding algorithm in `navigation.py` operates on a graph of `walkable_nodes` constructed from surveyed (x, y) coordinates read from the dataset. The graph construction function `build_graph()` creates an edge between two nodes if and only if they are orthogonally adjacent at a unit distance of 1 in grid space. Due to the manual nature of the physical survey, three specific corridor segments contained gaps where no reference coordinates were collected:

- Horizontal gap at $x = 27$ (central corridor, A-wing side).
- Horizontal gap at $x = 31$ (junction between wings).
- Vertical lift connection at $y = 2$ (lift lobby to main corridor).

These missing nodes physically severed the graph, rendering all nodes on one side of each gap unreachable from the other side. The A* algorithm correctly returns `None` when no path exists, and `app.py` maps this to an HTTP 404 response.

11.2.3 Solution: Heuristic Graph Bridging

The map generation script `build_floor_map.py` was updated with an automated gap-filling heuristic. After loading the surveyed coordinate set, the script programmatically inspects the corridor axis at $y = 8$ (the primary horizontal corridor shared by both wings) and automatically injects bridging nodes at the identified gap positions into the `walkable_nodes` array. This ensures full graph connectivity across both wings and the lift lobby without requiring additional physical survey work. Following this fix, all cross-wing and lift-to-corridor navigation requests resolved successfully.

11.3 Zone-Based Arrival Detection Failure

11.3.1 Observed Symptom

The backend was failing to trigger the "Arrival" proximity event accurately for Faculty Rooms and Laboratories. A user standing physically in front of a Faculty Room door would be reported as approximately 2 metres away, preventing the navigation system from registering the journey as complete.

11.3.2 Root Cause Analysis

To visually distinguish room categories on the 2D map UI and reflect physical parallelism in the rendered layout, the map assigns distinct artificial Y-coordinates to different room types:

- Main corridor nodes: $y = 8$
- Faculty Room nodes: $y = 10$
- Laboratory nodes: $y = 5$

The user's predicted position is always snapped to the nearest `walkable_node`, which lies on the main corridor ($y = 8$). The original `nearest_room_to_pos()` function in `navigation.py` computed arrival proximity using standard Euclidean distance:

$$d_{\text{Euclidean}}(u, r) = \sqrt{(r_x - u_x)^2 + (r_y - u_y)^2} \tag{6}$$

When a Faculty Room at $(x_r, 10)$ was compared against a user position at $(x_u, 8)$, the $\Delta y = 2$ offset introduced a permanent artificial baseline distance of 2 units, even when the user was directly in front of the door ($x_r = x_u$). This systematic inflation prevented the proximity threshold from ever being satisfied.

11.3.3 Solution: Axis-Specific Distance Calculation

The arrival detection logic was rewritten to apply a conditional distance metric. If a room's Y-coordinate differs from the main corridor axis (`corridor_y = 8`), the function discards the Y-axis entirely and evaluates proximity using only absolute X-axis deviation:

$$d(u, r) = \begin{cases} |r_x - u_x| & \text{if } r_y \neq \texttt{corridor\_y} \\ \sqrt{(r_x - u_x)^2 + (r_y - u_y)^2} & \text{otherwise} \end{cases} \tag{7}$$

```
if coord[1] != corridor_y:
dist = abs(coord[0] - pos[0])
else:
dist = math.hypot(coord[0] - pos[0], coord[1] - pos[1])
```

Listing 2: Axis-specific arrival distance in navigation.py

This aligns the mathematical distance calculation with physical reality: two positions share the same physical X-axis location regardless of their assigned UI Y-offsets. Following this fix, arrival detection operated correctly for all room types.

11.4 Room Layout Interleaving Discrepancy

11.4.1 Observed Symptom

The spatial ordering of specific rooms on the rendered map did not reflect their physical locations on the floor. Notably, Discussion Area A-417 was rendered at the far end of the A-wing corridor, whereas it physically sits between two distinct laboratory clusters. Correspondingly, the A* pathfinder was routing users to geometrically incorrect destination nodes, causing them to stop at the wrong doorway.

11.4.2 Root Cause Analysis

A transcription error during the initial coordinate mapping phase caused the `representative_x` values for two pairs of nodes — specifically A-414/A-417 and B-414/B-417 — to be swapped in the `room_mapping.json` configuration file. Because the A* algorithm uses these coordinates as its destination nodes, a swapped coordinate meant the pathfinder was directing users to the physical location of a different room entirely.

11.4.3 Solution: Map Registration Overhaul

The `room_mapping.json` matrix was fully restructured and verified against a physical walk-through of the floor. The corrected spatial ordering enforces the following alternating interleaving pattern that matches the physical room sequence observed on-site:

Meeting Room → Lab → Lab → Discussion Room → Lab → Lab → Discussion Room → Lab

This ensures that every destination node registered in the A* pathfinder exactly corresponds to the physical doorway the user intends to reach, restoring correct end-to-end navigation for all affected rooms.

11.5 Accuracy and Limitations

The primary accuracy constraint of the KNN fingerprinting approach is its sensitivity to the density and uniformity of the training radio map. Field testing revealed a critical asymmetry: A-wing reference points contained only 8–9 AP observations per coordinate, while B-wing points contained 12–16. This caused the model to produce 23-metre localization errors in the A-wing

(the "teleportation" glitch) because the KNN distance metric heavily penalised the ∼42 live APs absent from the sparse training vectors. This was resolved by the Top-12 AP truncation filter described in Section 10.1. Regarding temporal accuracy, position smoothing variables (`lastX`, `lastY`) were explicitly removed in the final production build to eliminate perceived UI lag and input latency, as confirmed in `MainActivity.kt`. This means individual scan results may exhibit single-scan jitter of 1–2 grid units. The `wifiManager.startScan()` API is deprecated in Android 9 and later versions, and is subject to OS-level throttle limiting (typically four scans per 2-minute window for background apps). The application falls back to cached scan results when a fresh scan is denied by the OS, which may cause the reported position to be stale by up to one throttle window. During field testing, three structural gaps were identified in the surveyed coordinate set at $x = 27$, $x = 31$, and the vertical lift connection at $y = 2$. These gaps physically severed the graph, preventing the A* algorithm from finding cross-wing paths and causing HTTP 404 navigation errors. The `build_floor_map.py` script was subsequently updated with a heuristic gap-filling pass that automatically injects bridging nodes at these positions during map generation, restoring full graph connectivity. See Section 10.2 for the complete root cause analysis. Furthermore, to achieve readable visual separation between room categories on the 2D canvas, the map assigns distinct artificial Y-coordinates to different room types: Faculty Rooms are placed at $y = 10$, Laboratories at $y = 5$, and the main walkable corridor at $y = 8$. These Y-offsets are purely cosmetic and have no physical meaning; however, they required a corresponding correction to the arrival detection distance metric (see Section 10.3). **Training Data Density Asymmetry:** The A-wing radio map was surveyed with significantly fewer AP observations per reference point (8–9) compared to the B-wing (12–16). This asymmetry caused the production KNN model to produce large localization errors specifically in the A-wing. The short-term fix is the Top-12 AP truncation filter in `app.py`; the long-term solution would be to re-survey the A-wing with denser reference point coverage. **Artificial UI Y-Offsets Leaking into Physics:** The decision to assign different Y-coordinates to different room types for visual clarity (Faculty at $y = 10$, Labs at $y = 5$) created a tight coupling between the rendering layer and the arrival detection logic in `navigation.py`. This is a design fragility: any future change to the visual layout constants must be accompanied by a matching update to the arrival distance formula, or arrival detection will silently regress. **Manual Coordinate Transcription Risk:** The coordinate swap for rooms A-414/A-417 and B-414/B-417 demonstrated that `room_mapping.json` is currently edited by hand with no automated validation against physical survey records. A diffing or checksum tool against the ground-truth survey dataset would prevent similar transcription errors in future mapping updates. **Pre-processing note:** Prior to passing the BSSID dictionary to `build_feature_vector`, the `localize()` handler applies a truncation step that retains only the top 12 strongest APs by RSSI value. This normalises the feature density of the live scan to match the training data distribution and prevents the KNN model from producing spurious distance penalties caused by excess weak APs absent from the training fingerprints. See Section 10.1 for the full technical analysis.

12 PROJECT SCALE AND MODEL PERFORMANCE

This section quantifies the deployed system relative to the previous LOCUS baseline, summarises the trained KNN model's measured performance, and documents the methodology used for hyperparameter selection and floor map generation.

12.1 Scale Comparison: 2025 Baseline vs. 2026 Deployment

Metric	LOCUS 2025 baseline	Locus 2026 deployment
Walkable nodes	57	**122**
Mapped rooms	18	**40**
Unique BSSIDs (feature dim)	33	**135**
Training fingerprints	56	**125**
Floor layout layers	1 (corridor only)	3 (labs y=5, corridor y=8, offices y=10)
Wings covered	A, B	A (negative x), B (positive x)

The 2026 deployment more than doubles every quantitative dimension of the radio map, introducing structured y-axis stratification for room categories and an explicit lift landmark anchored at the origin $(0, 0)$.

12.2 Model Performance Summary

The production model is the artifact persisted at `backend/knn_localization_model.joblib`. Its training and validation metrics are summarised below.

Metric	Value
Algorithm	`KNeighborsRegressor` (multi-output)
Best k (neighbours)	5
Weights	distance
Distance metric	manhattan
Mean Absolute Error (LOO)	1.04 grid units
Mean Euclidean Error (LOO)	1.77 grid units
Median Euclidean Error (LOO)	1.47 grid units
Cross-validation strategy	Leave-One-Out (LOO)
Training points	125 (from `coordinates.json`)
Unique BSSIDs (features)	135

The multi-output `KNeighborsRegressor` predicts x and y jointly, which preserves the spatial correlation between the two coordinates relative to training a separate model per axis.

12.3 Hyperparameter Search Methodology

The hyperparameter search is implemented in `tune_knn.py`. The search grid is exhaustive over neighbour count, weighting scheme, and distance metric:

- `n_neighbors`: {1, 2, 3, 4, 5, 7}
- `weights`: {`uniform`, `distance`}
- `metric`: {`euclidean`, `manhattan`, `chebyshev`}

This 36-cell grid is evaluated under **Leave-One-Out (LOO)** cross-validation. With only 125 training points, LOO is the preferred strategy: each fold trains on $N - 1 = 124$ points and validates on the single held-out point, repeated 125 times. This yields an almost-unbiased

generalization estimate while wasting no training data, which would not be possible with k-fold CV at small N. The scoring metric is `neg_mean_squared_error`; `GridSearchCV` is run with `n_jobs=-1` for parallel evaluation. After the search, the best configuration is retrained on the full 125-point dataset and serialised together with the BSSID feature ordering. The full code listings for these stages appear in Appendix A.1–A.3.

12.4 Floor Map Generation Pipeline

The floor map is generated by `build_floor_map.py`, which consumes `coordinates.json` (raw surveyed grid points) and `room_mapping.json` (room metadata) and emits `floor_map_new.json`. Two non-trivial transformations occur:

1. **Gap-fill bridging** (deterministic): three nodes — $(27, 8)$, $(31, 8)$, and $(0, 2)$ — are unconditionally appended to the `walkable_nodes` array if absent. These bridge the corridor gaps documented in Section 10.2 and ensure the A* adjacency graph is fully connected across both wings and the lift lobby.

2. **Y-axis stratification by room type:** for every room declared in `room_mapping.json`, the y-coordinate is derived from the `type` field rather than entered manually. Offices and meeting rooms receive $y = 10$ (above the corridor); labs and discussion areas receive $y = 5$ (below). The x-coordinate is taken from each room's `representative_x`. This data-driven assignment eliminates the manual transcription class of bug documented in Section 10.4. The lift is anchored at $(0, 0)$.

The full code listings for both transformations appear in Appendix A.4–A.5.

13 BUILD, DEPLOYMENT AND OPERATIONAL DEBUGGING

13.1 Android Manifest Permissions

The following permissions are declared in `AndroidManifest.xml` and are all required for correct end-to-end operation:

Permission	Purpose
`ACCESS_WIFI_STATE`	Read WiFi adapter state.
`CHANGE_WIFI_STATE`	Trigger `startScan()`.
`ACCESS_FINE_LOCATION`	Required to read scan results on Android 6+.
`ACCESS_COARSE_LOCATION`	Required alongside fine location at runtime.
`ACCESS_NETWORK_STATE`	Determine LAN connectivity for backend reachability.
`INTERNET`	Open HTTP socket to the Flask backend.

The application sets `android:usesCleartextTraffic="true"` to permit unencrypted HTTP traffic to the local Flask backend over LAN. On Android 13+ devices (`targetSdk = 34`), it is recommended to additionally declare `android.permission.NEARBY_WIFI_DEVICES`; without it, `WifiManager.startScan()` may return empty results even when `ACCESS_FINE_LOCATION` is granted.

13.2 Gradle Build Configuration

- `compileSdk`: 34 • `targetSdk`: 34 • `minSdk`: 26 (Android 8.0)

- Kotlin JVM target: 1.8 • Java source/target compatibility: 1.8
- `buildFeatures.viewBinding = true`
- `namespace`: `com.wn.indoornavigation` • `applicationId`: `com.wn.indoornavigation`

13.3 Dependency Versions

- `androidx.core:core-ktx:1.12.0`
- `androidx.appcompat:appcompat:1.6.1`
- `com.google.android.material:material:1.11.0`
- `androidx.constraintlayout:constraintlayout:2.1.4`
- `com.squareup.retrofit2:retrofit:2.9.0`
- `com.squareup.retrofit2:converter-gson:2.9.0`
- `com.squareup.okhttp3:logging-interceptor:4.12.0`
- `org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3`
- `androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0`
- `androidx.lifecycle:lifecycle-runtime-ktx:2.7.0`

The Python backend has the following runtime dependencies (no `requirements.txt` is checked in; install via `pip`):

```
pip install flask flask-cors scikit-learn joblib numpy
```

Listing 3: Backend dependency install

13.4 Operational Troubleshooting Matrix

Symptom	Likely Cause and Resolution
Flask: "Address already in use" on port 5000 (macOS)	macOS AirPlay Receiver occupies port 5000 by default. Either change `app.run(port=5001)` in `app.py` or disable AirPlay Receiver under System Settings → General → AirDrop & Handoff.
Android: "Could not reach backend"	`BASE_URL` in `ApiClient.kt` does not match the laptop's current LAN IP. Run `ipconfig` (Windows) or `ifconfig` / `ipconfig getifaddr en0` (macOS), then update the constant and rebuild.
`startScan()` returns empty list	`ACCESS_FINE_LOCATION` denied at runtime, or `NEARBY_WIFI_DEVICES` missing on Android 13+. Grant Location permission for the app and add the manifest entry for Android 13+.
Confidence is consistently low	Fewer than 2 of the 135 known BSSIDs are visible, or `bssids.json` in `backend/` is stale. Move toward the centre of the corridor or re-copy `bssids.json` from project root.

Wrong location after retraining	`knn_localization_model.joblib` and `bssids.json` in `backend/` are out of sync. Always copy *both* files atomically from the project root after running `tune_knn.py`.
HTTP 404 on `/navigate`	Snapped position falls in a graph component disconnected from the destination. Verify `floor_map.json` contains the gap-fill nodes $(27, 8)$, $(31, 8)$, $(0, 2)$. Regenerate via `python build_floor_map.py` if missing.
Gradle sync fails in Android Studio	Network outage during dependency download, or stale caches. Use File → Invalidate Caches and Restart.

13.5 Strict Dependency Warning

CRITICAL: The file `bssids.json` defines the **Feature Vector Layout** of the production KNN model. **Never** manually edit this file. It is auto-generated by `tune_knn.py` and persisted in lockstep with `knn_localization_model.joblib`. The order of the 135 BSSIDs must exactly match the column order used during training; reordering, addition, or deletion will silently corrupt every localization prediction without raising any exception.

After every run of `tune_knn.py`, copy both `knn_localization_model.joblib` and `bssids.json` from the project root into the `backend/` folder atomically before restarting `app.py`.

14 APPENDIX: VERBATIM CODE LISTINGS

This appendix collects the canonical implementation excerpts referenced throughout the body. Each listing is captioned with the source file and approximate line range.

A.1 `tune_knn.py` (lines 18–30): BSSID vocabulary and feature vector construction

```
def preprocess_data(data):
all_bssids = set()
for point in data:
all_bssids.update(point.get('rssi', {}).keys())
all_bssids = list(all_bssids)
X, y = [], []
for point in data:
features = [point.get('rssi', {}).get(bssid, -100)
for bssid in all_bssids]
X.append(features)
y.append([point['x'], point['y']])
return np.array(X), np.array(y), all_bssids
```

A.2 `tune_knn.py` (lines 48–70): hyperparameter grid and Leave-One-Out search

```
param_grid = {
'n_neighbors': [1, 2, 3, 4, 5, 7],
'weights':     ['uniform', 'distance'],
'metric':      ['euclidean', 'manhattan', 'chebyshev'],
}
```

```
loo = LeaveOneOut()
grid = GridSearchCV(
KNeighborsRegressor(),
param_grid,
cv=loo,
scoring='neg_mean_squared_error',
n_jobs=-1,
refit=True
)
grid.fit(X, y)
```

A.3 `tune_knn.py` (lines 110–116): final retrain and persistence

```
final_model = KNeighborsRegressor(**best_params)
final_model.fit(X, y)
joblib.dump(final_model, 'knn_localization_model.joblib')
with open('bssids.json', 'w') as f:
json.dump(bssids, f)
```

A.4 `build_floor_map.py` (lines 17–25): gap-fill bridge nodes

```
# Fill corridor gaps: x=27 and x=31 at y=8
# Also fill vertical gap: y=2 at x=0
gap_fills = [[27, 8], [31, 8], [0, 2]]
for gf in gap_fills:
key = tuple(gf)
if key not in seen:
seen.add(key)
walkable.append(gf)
print(f'  Added gap-fill node: {gf}')
```

A.5 `build_floor_map.py` (lines 32–49): faculty/lab y-axis assignment by room type

```
FACULTY_TYPES = {'Office'}
LAB_TYPES = {'Lab', 'Discussion Area'}
MEETING_TYPES = {'Meeting Room'}
for room_id, info in room_data['rooms'].items():
rx = info['representative_x']
room_type = info['type']
if room_type in FACULTY_TYPES or room_type in MEETING_TYPES:
ry = 10   # above the corridor (y=8)
else:
ry = 5    # below the corridor (labs)
room_map[room_id] = [rx, ry]
```

A.6 `backend/navigation.py` (lines 22–37): `build_graph` 4-directional adjacency

```
def build_graph(walkable_nodes):
node_set = {tuple(n) for n in walkable_nodes}
graph = {node: [] for node in node_set}
for (x, y) in node_set:
for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
neighbour = (x + dx, y + dy)
if neighbour in node_set:
graph[(x, y)].append((neighbour, 1))
return graph
```

A.7 backend/navigation.py (lines 44–46): Manhattan heuristic

```
def heuristic(a, b):
"""Manhattan distance -- optimal for 4-directional grids."""
return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

A.8 backend/navigation.py (lines 135–139): axis-specific arrival distance

```
if coord[1] != corridor_y:
dist = abs(coord[0] - pos[0])           # x-only for off-axis rooms
else:
dist = math.hypot(coord[0] - pos[0],
coord[1] - pos[1])
```

A.9 backend/navigation.py (lines 179–192): direction collapse with landmark callouts

```
for i in range(1, len(path)):
dx = path[i][0] - path[i-1][0]
dy = path[i][1] - path[i-1][1]
direction = (dx, dy)
if path[i] in waypoint_labels and current_dir is not None:
instructions.append(
f"Go {DIRECTION_NAMES[current_dir]} for {step_count} m")
instructions.append(f"Pass {waypoint_labels[path[i]]}")
step_count = 0
current_dir = None
```

A.10 backend/app.py (localize route): Top-12 RSSI truncation & feature extraction

```
# Truncate to Top 12 strongest APs to match training feature density
sorted_bssids = sorted(rssi_dict.items(), key=lambda x: x[1], reverse=True)
top_rssi_dict = dict(sorted_bssids[:12])

# Build feature vector and predict
features = [top_rssi_dict.get(bssid, -100) for bssid in BSSIDS]
features_array = np.array(features).reshape(1, -1)
```

A.11 backend/app.py (lines 138–144): confidence banding and zone arrival

```
known_seen = sum(1 for b in rssi_dict if b in BSSIDS)
confidence = ("high" if known_seen >= 5
else "medium" if known_seen >= 2
else "low")
snapped_pos = (snapped[0], snapped[1])
nearby_room = nav.nearest_room(snapped_pos)
```

A.12 WifiScanner.kt (lines 56–84): broadcast receiver and strongest-signal dedupe

```
val receiver = object : BroadcastReceiver() {
override fun onReceive(ctx: Context, intent: Intent) {
context.unregisterReceiver(this)
val results = wifiManager.scanResults
val bssidMap = mutableMapOf<String, Int>()
for (result in results) {
val mac = result.BSSID.lowercase()
val rssi = result.level
if (!bssidMap.containsKey(mac) || rssi > bssidMap[mac]!!)
bssidMap[mac] = rssi
}
onResult(bssidMap)
}
}
```

A.13 `FloorMapView.kt` (lines 195–200): `toScreen` grid-to-pixel transform with y-axis inversion

```
private fun toScreen(gx: Float, gy: Float): PointF {
return PointF(
offsetX + gx * CELL_PX * scaleFactor,
offsetY - gy * CELL_PX * scaleFactor   // y-axis flip
)
}
```

A.14 `FloorMapView.kt`: Auto-pan camera centering

```
/** Pan the camera to center on the current user position */
fun centerOnUser() {
userPosition?.let { (ux, uy) ->
offsetX = width / 2f - ux * CELL_PX * scaleFactor
offsetY = height / 2f + uy * CELL_PX * scaleFactor
invalidate()
}
}
```

A.15 `network/ApiClient.kt` (lines 87–94): Retrofit singleton lazy initialiser

```
object ApiClient {
val service: ApiService by lazy {
Retrofit.Builder()
.baseUrl(BASE_URL)
.addConverterFactory(GsonConverterFactory.create())
.build()
.create(ApiService::class.java)
}
}
```

15 GLOSSARY

- **A* (A-Star):** A graph traversal and path search algorithm heavily utilized in computer science due to its performance and accuracy, used here to find the shortest indoor walking path.

- **BSSID (Basic Service Set Identifier):** The MAC address of the wireless access point acting as a unique beacon identifier.
- **Fingerprinting:** A localization technique that involves surveying the radio frequency landscape at known points (creating a radio map) and later comparing real-time signals against this database.
- **KNN (K-Nearest Neighbors):** A supervised machine learning algorithm used by the backend to find the K closest offline fingerprints matching the online scan to estimate position.
- **Radio Map:** The database of signals collected during the calibration phase, stored within `survey.json` and embedded in the trained model.
- **RSSI (Received Signal Strength Indicator):** An estimated measure of power level received by the client antenna, measured in negative decibel-milliwatts (dBm). Lower negative numbers indicate stronger signals.
- **ViewBinding:** An Android feature that allows you to more easily write code that interacts with views, replacing older `findViewById` approaches in modern Android development.
- **Feature Density:** The number of non-default (i.e., non--100 dBm) entries in a fingerprint feature vector, corresponding to the number of APs detected at a reference point during the survey phase. Asymmetric feature density between training regions is a primary source of KNN localization error.
- **Graph Bridging:** The programmatic injection of synthetic walkable nodes into the floor graph to fill physical gaps in the surveyed coordinate set, ensuring full connectivity for the A* pathfinding algorithm.
- **Arrival Detection:** The zone-based proximity check that determines whether the user's estimated position is sufficiently close to a room's registered coordinate to be declared as "arrived". In this system, arrival is evaluated using an axis-specific distance metric to account for artificial UI Y-offsets in the room layout.
- **Top-K Truncation:** A pre-processing step applied to the live WiFi scan that retains only the K strongest APs by RSSI before constructing the KNN feature vector, used here with $K = 12$ to match training data density.